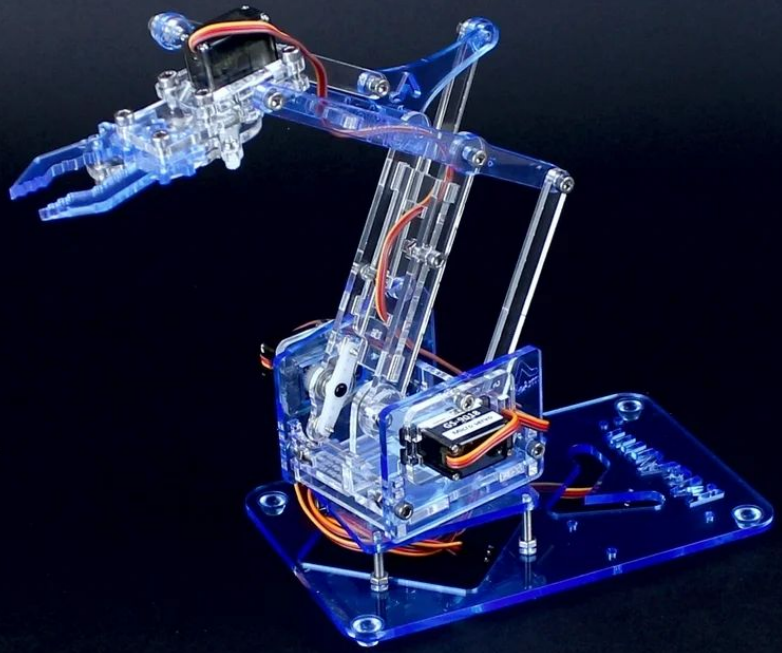
MeArm



Links

Official MeArm resources

https://shop.mearm.com/pages/resources-and-files

Model

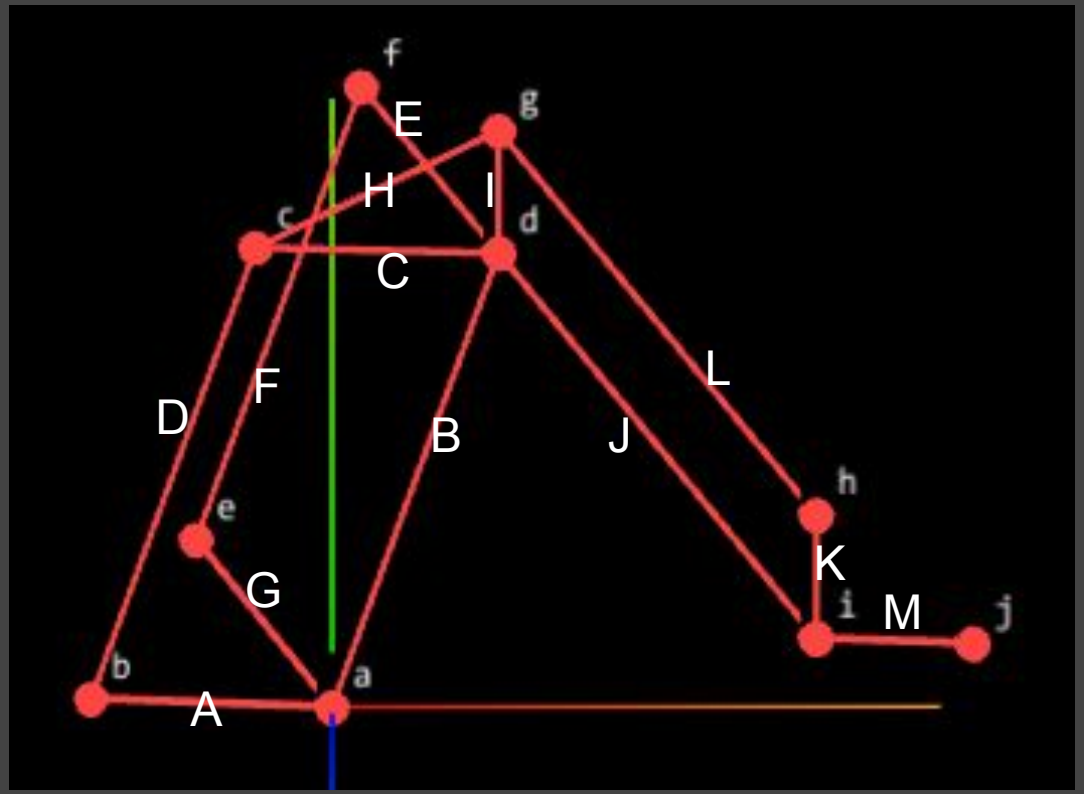
https://sketchfab.com/3d-models/mearm-v10-robot-arm-8d8d0a43e0ba4ee5808e703da62fff28

Simulator

https://gerardomunoz.github.io/embedded/virtual/mearm_seq.html

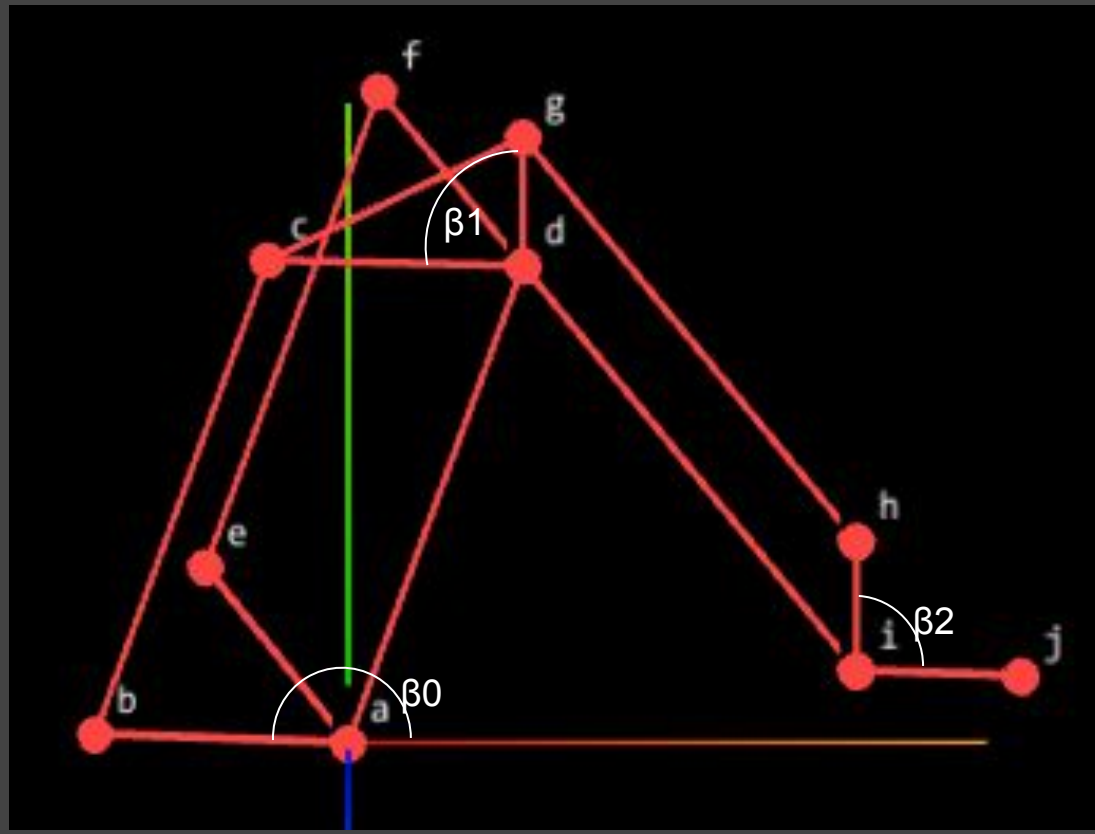
MeArm Links and Joins

Is M parallel to A?

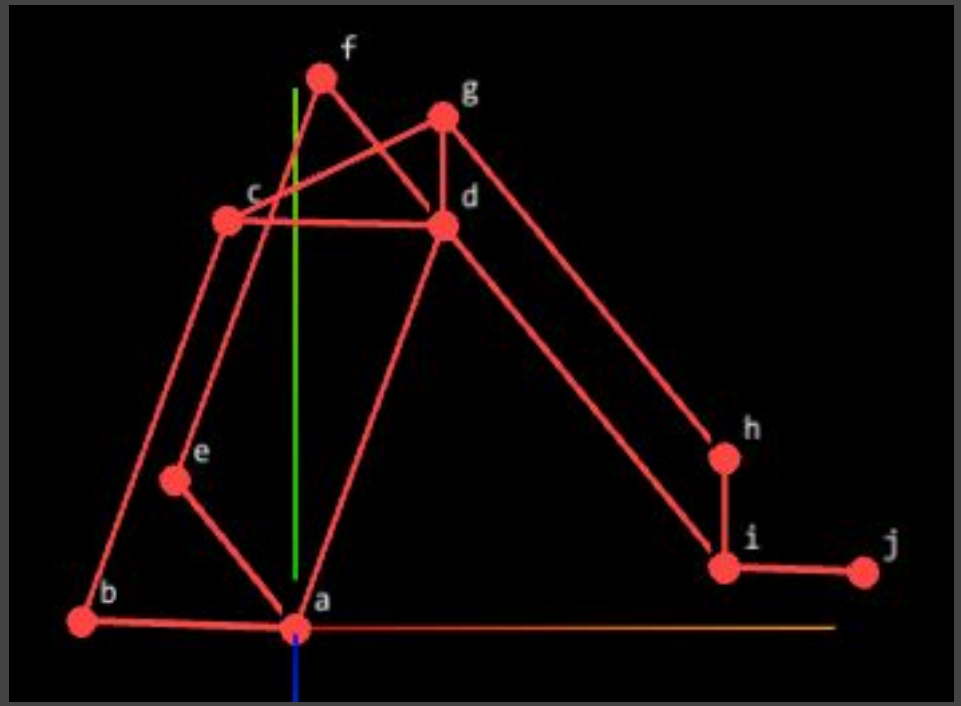


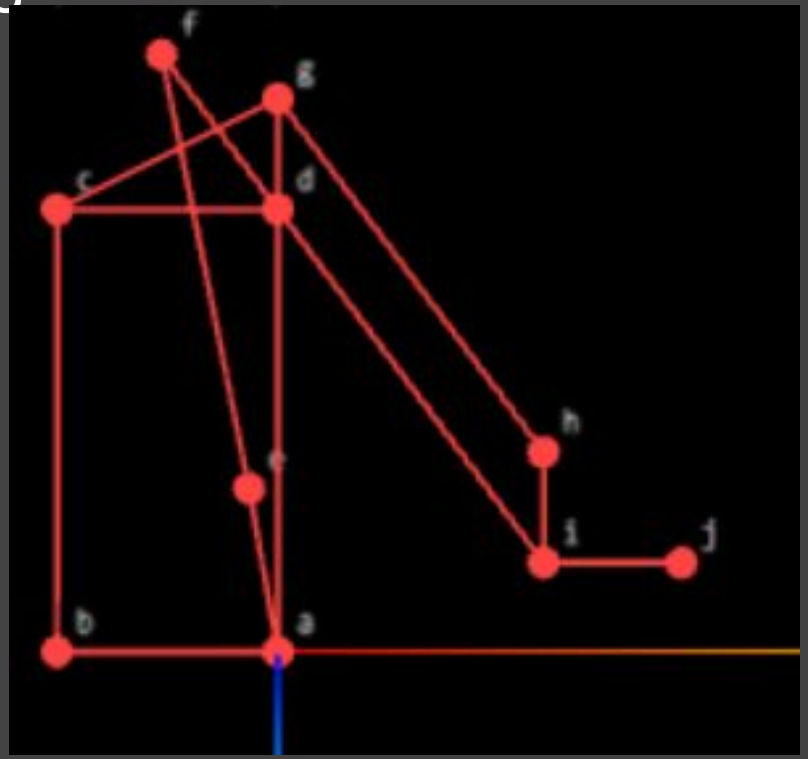
MeArm Constant angles

What would happen if β0 changed?



Parallelogram vs Four-bar linkage





Four-bar linkage, examples

https://www.youtube.com/watch?v=ohPwT1TYvqE&list=PL3p-xXY-sN6GKWr1vW_iW8NHEkeZBoL3T&index=133

https://youtu.be/vzM_-Rr5mtw?list=PL3p-xXY-sN6GKWr1vW_iW8NHEkeZBoL3T&t=431

https://www.youtube.com/watch?v=5ecQ2bxHRjI

https://www.youtube.com/watch?v=NM4q-f68TIY

https://www.youtube.com/watch?v=nhd0NdZDjWw

https://www.youtube.com/watch?v=JFmNAI-4cQo&list=PL3p-xXY-sN6GKWr1vW_iW8NHEkeZBoL3T&index=63

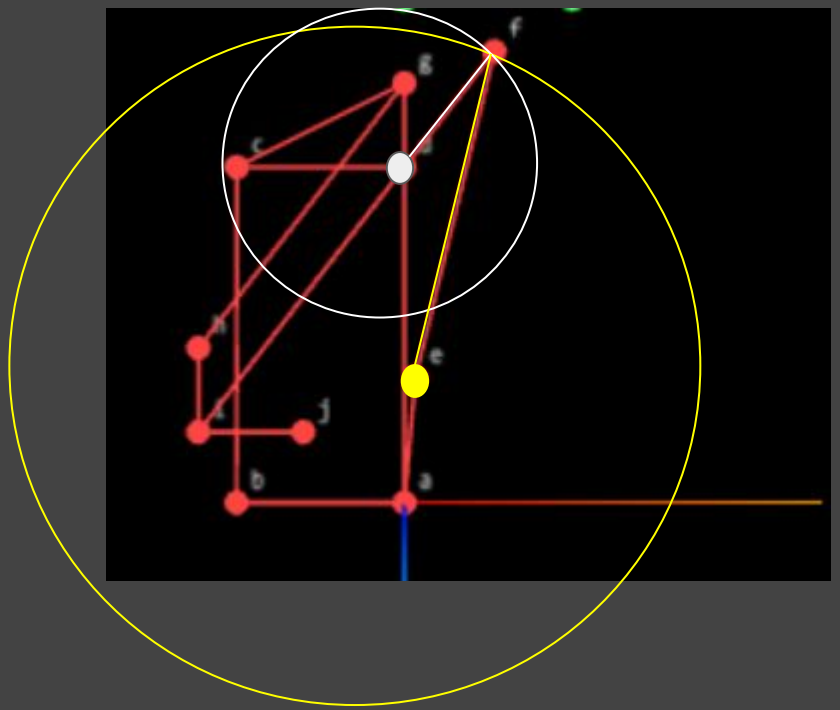
https://www.youtube.com/watch?v=NdIk5hIsBmg&list=PL3p-xXY-sN6GKWr1vW_iW8NHEkeZBoL3T&index=57

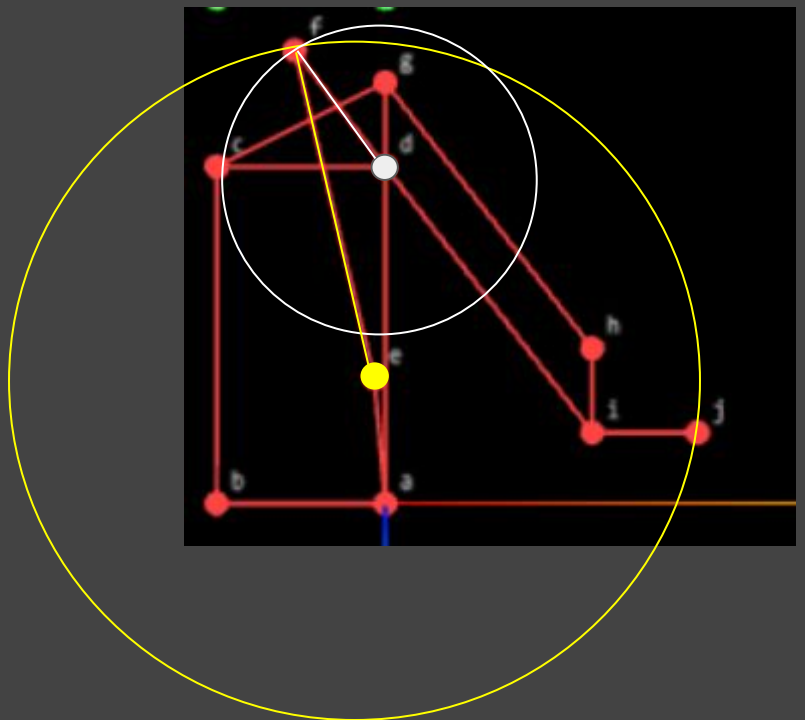
https://www.youtube.com/watch?v=74T0LF2I5Ck&list=PL3p-xXY-sN6GKWr1vW_iW8NHEkeZBoL3T&index=44

https://www.youtube.com/watch?v=VbSL5W8Ua6M&list=PL3p-xXY-sN6GKWr1vW_iW8NHEkeZBoL3T&index=135

https://www.youtube.com/watch?v=7hBNs4Xqdy4

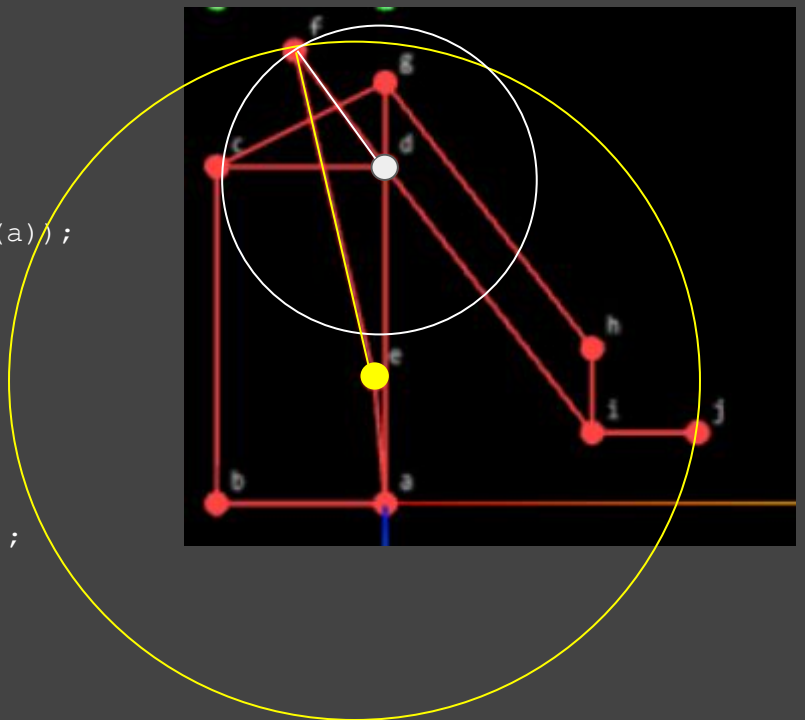
Four-bar linkage, two solutions





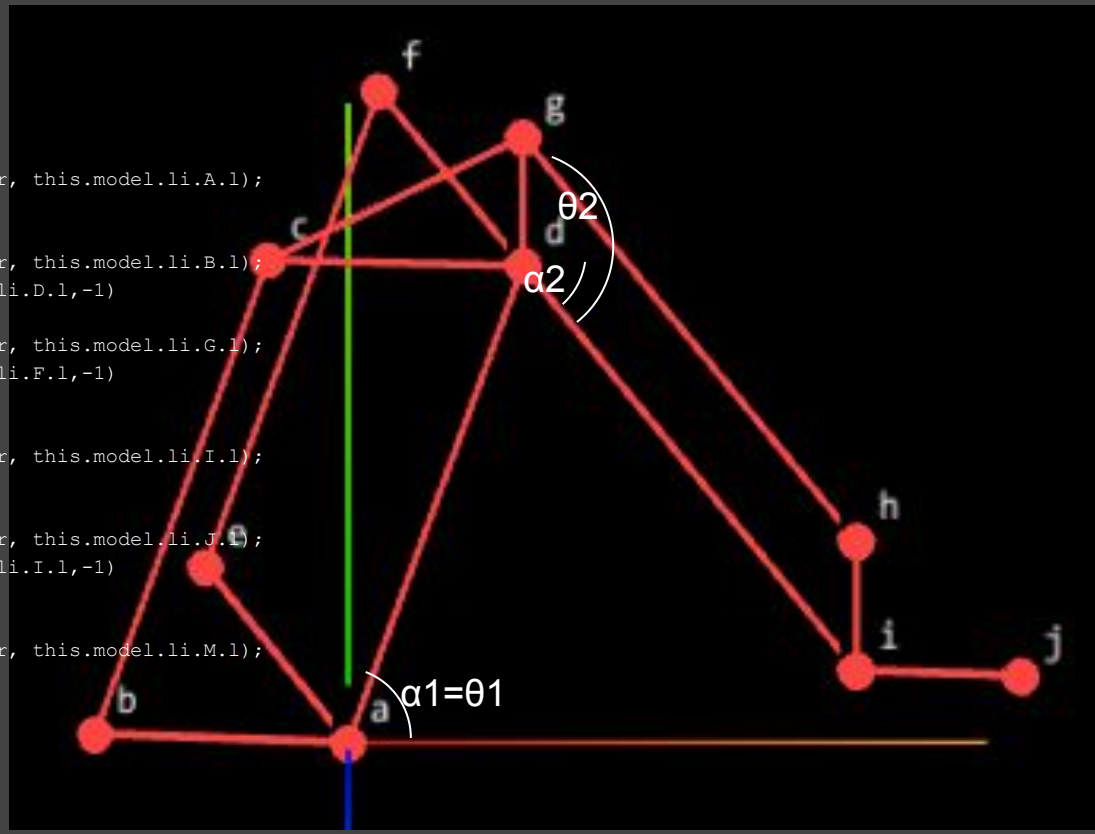
Four-bar linkage, algorithm

```
function fbl(A, D, B, L2, L3, elbow = 1) {
    const AB = new THREE.Vector3().subVectors(B, A);
    const BD = new THREE.Vector3().subVectors(D, B);
    const d = BD.length();
    const ex = BD.clone().normalize();
    const a = (L2 * L2 - L3 * L3 + d * d) / (2 * d);
    const h2 = L2 * L2 - a * a;
    if (h2 < 0) return null;
    const h = Math.sqrt(h2);
    const P = B.clone().add(ex.clone().multiplyScalar(a));
    const normal = new THREE.Vector3()
        .crossVectors(AB, BD)
        .normalize();
    const ey = new THREE.Vector3()
        .crossVectors(normal, ex)
        .normalize();
    return P.clone().add(ey.multiplyScalar(elbow * h));
}
```

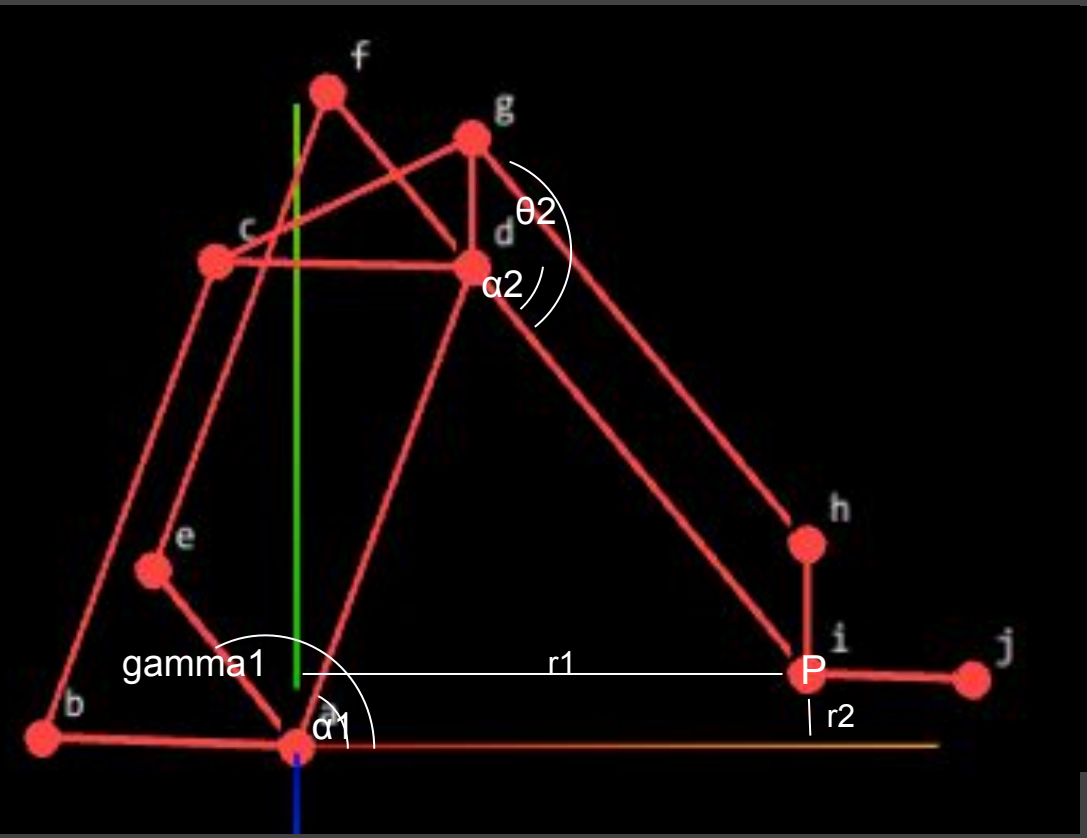


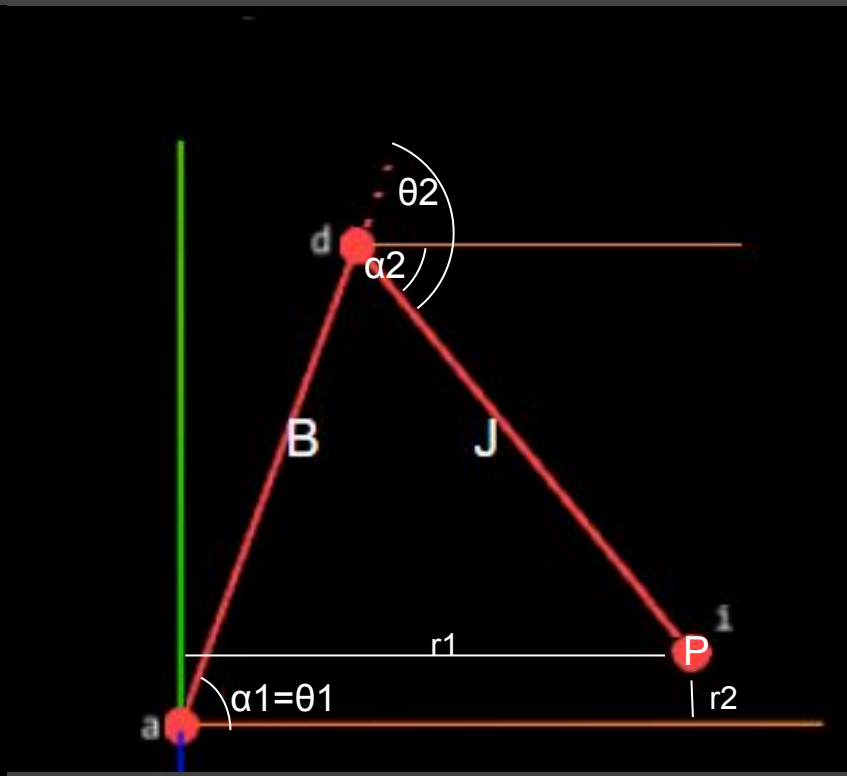
Direct Kinematics

```
computePoints_deg(alphas, ctes) {
    const pts = {};
    // --- helpers ---
    const rotZ = (angle) => new THREE.Vector3(Math.cos(angle), Math.sin(angle), 0);
    const angleBetween = (p1, p2) => {
        const v = new THREE.Vector3().subVectors(p1, p2);
        return Math.atan2(v.y, v.x);
    };
    // --- base point ---
    pts.a = new THREE.Vector3(0, 0, 0);
        const dir = rotZ(ctes.beta1);
        pts.b = new THREE.Vector3().copy(pts.a).addScaledVector(dir, this.model.li.A.l);
        console.log('b',dir, this.model.li.A.l,pts.a,pts.b)
        const dir = rotZ(alphas.alpha1);
        pts.d = new THREE.Vector3().copy(pts.a).addScaledVector(dir, this.model.li.B.l);
        pts.c= fbl(pts.a,pts.b,pts.d,this.model.li.C.l,this.model.li.D.l,-1)
        const dir = rotZ(alphas.alpha2);
        pts.e = new THREE.Vector3().copy(pts.a).addScaledVector(dir, this.model.li.G.l);
        pts.f= fbl(pts.a,pts.e,pts.d,this.model.li.E.l,this.model.li.F.l,-1)
        const ang = angleBetween(pts.d, pts.c) + ctes.beta2;
        const dir = rotZ(ang);
        pts.g = new THREE.Vector3().copy(pts.d).addScaledVector(dir, this.model.li.I.l);
        const ang = angleBetween(pts.d, pts.f);
        const dir = rotZ(ang);
        pts.i = new THREE.Vector3().copy(pts.d).addScaledVector(dir, this.model.li.J.l);
        pts.h= fbl(pts.d,pts.i,pts.g,this.model.li.J.l,this.model.li.I.l,-1)
        const ang = angleBetween(pts.i, pts.h) + ctes.beta3;
        const dir = rotZ(ang);
        pts.j = new THREE.Vector3().copy(pts.i).addScaledVector(dir, this.model.li.M.l);
    return pts;
}
```



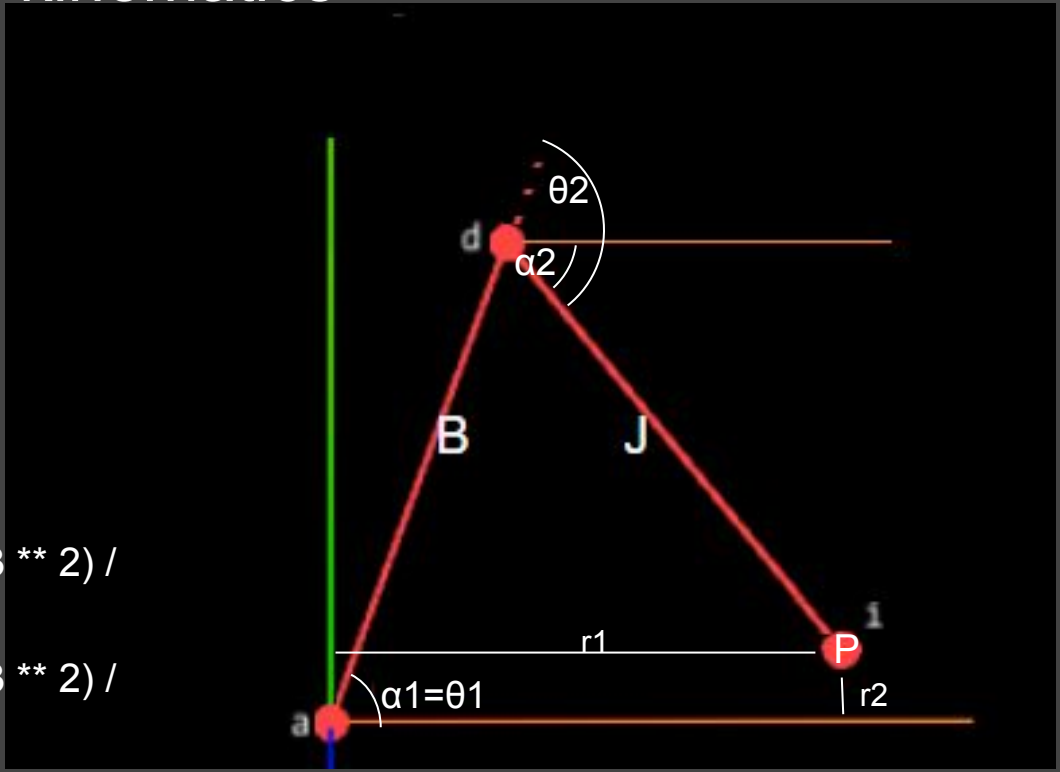
Inverse kinematics, two steps

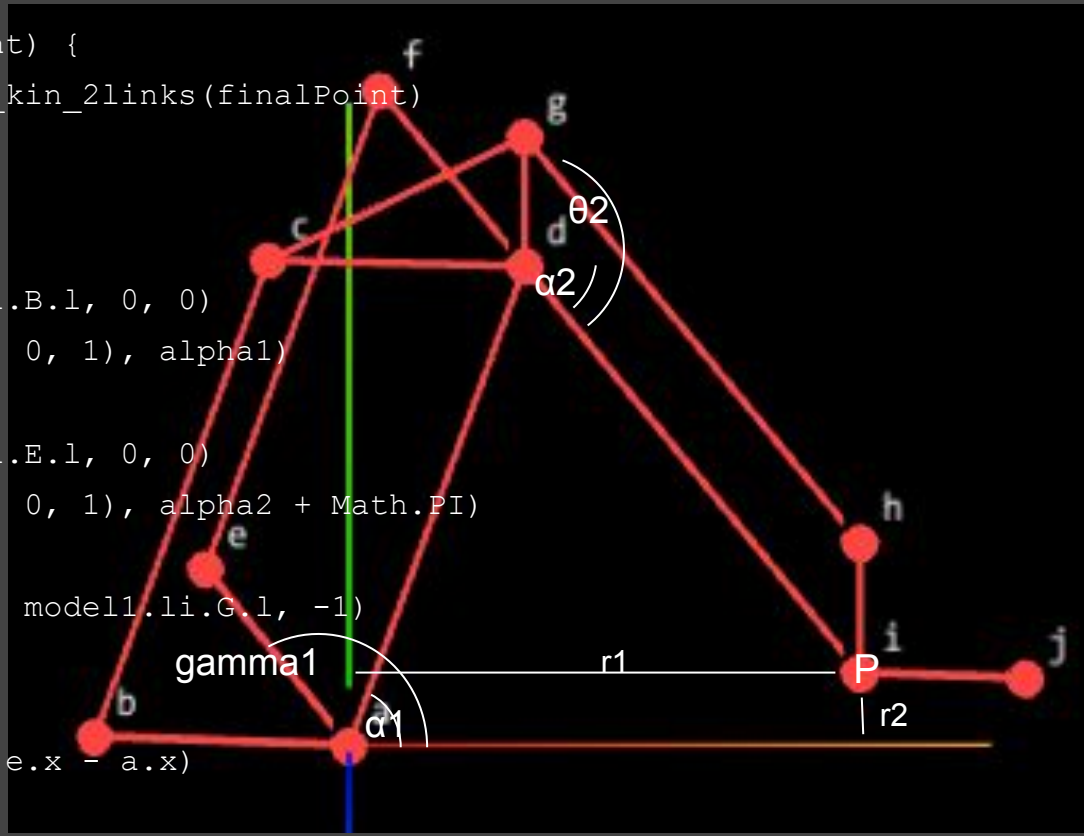




Two links, two joins inverse kinematics

```
function inv_kin_2links_rad(P) {
    const a1 = 0
    const a2 = model1.li.B.l
    const a3 = model1.li.J.l
    const x03 = P.x;
    const z03 = P.y;
    const y03 = -P.z;
    const alpha0 = Math.atan2(y03, x03);
    const r1 = Math.sqrt(x03 ** 2 + y03 ** 2);
    const r2 = -(z03 - a1);
    const phi2 = Math.atan2(r2, r1);
    const r3 = Math.sqrt(r1 ** 2 + r2 ** 2);
    const phi1 = Math.acos((a3 ** 2 - a2 ** 2 - r3 ** 2) /
                                    (-2 * a2 * r3));
    const alpha1 = phi1 - phi2;
    const phi3 = Math.acos((r3 ** 2 - a2 ** 2 - a3 ** 2) /
                                    (-2 * a2 * a3));
    const alpha2 = phi3 - Math.PI / 2;
    return [alpha0, alpha1, alpha2];
}
```



MeArm inverse kinematics

```
function mearm_inverseKinematics(finalPoint) {
    const [alpha0, alpha1, alpha2] = inv_kin_2links(finalPoint)
    const gamma0 = alpha0
    const gamma1 = alpha1
    const a = new THREE.Vector3(0, 0, 0)
    const d = new THREE.Vector3(model1.li.B.l, 0, 0)
    d.applyAxisAngle(new THREE.Vector3(0, 0, 1), alpha1)
    d.add(a)
    const f = new THREE.Vector3(model1.li.E.l, 0, 0)
    f.applyAxisAngle(new THREE.Vector3(0, 0, 1), alpha2 + Math.PI)
    f.add(d)
    const e = fbl(d, a, f, model1.li.F.l, model1.li.G.l, -1)


    const gamma2 = Math.atan2(e.y - a.y, e.x - a.x)
    return [gamma0, gamma1, gamma2]
}
```